HTTP 완벽 가이드 · Week 7

캐시 (2)

캐시 제어와 정책 설계 · 7장 · p.209 ~ 226

정우인

PART 1

캐시 제어

7.9 · p.209 ~

만료까지 얼마나 캐시할지, 서버가 정한다

문서가 만료되기 전까지 **얼마나 오래 캐시될 수 있는지**를 서버가 설정하는 여러 방법

1 `Cache-Control: no-store`

사본을 아예 저장하지 않음

2 `Cache-Control: no-cache`

저장은 OK · 쓰기 전 매번 재검사

3 `Cache-Control: must-revalidate`

만료되면 반드시 재검사

4 `Cache-Control: max-age`

신선한 기간을 초 단위로

5 `Expires`

만료되는 절대 날짜

6 **(아무것도 안 줌)**

캐시가 휴리스틱으로 직접 결정

→ Cache-Control 지시자 4가지 + Expires + 아무것도 없을 때의 휴리스틱

no-store와 no-cache — 검증 안 된 사본을 막는다

공통 목적 캐시가 **검증되지 않은 캐시된 객체**로 응답하는 것을 막음

1 Cache-Control: no-store

저장	✕	사본을 만들지 않음
제공	✕	매번 서버에서 새로 받음

2 Cache-Control: no-cache

저장	○	로컬 캐시 저장소에 저장됨
제공	△	서버와 재검사한 뒤에만

하위 호환 `Pragma: no-cache` — HTTP/1.0+ 와의 하위 호환을 위해 HTTP/1.1에 포함

→ no-cache는 '캐시 금지'가 아니라 `Do not serve from cache without revalidation`

max-age — 몇 초 동안 신선한가

max-age = 문서가 서버에서 온 뒤로 **신선하다고 간주되는 시간** (초)

1 Cache-Control: max-age=3484539

모든 캐시에 적용

- ✓ 브라우저 개인
- ✓ 프록시 공용
- ✓ CDN 공용

2 Cache-Control: s-maxage=3484539

공용 캐시에만 적용 · 동작은 max-age와 같음

- ✗ 브라우저 개인
- ✓ 프록시 공용
- ✓ CDN 공용

max-age=0 서버가 캐시에게 '매 접근마다 캐시하지 말거나, 매번 리프레시하라'고 요청

→ **받은 시점부터 N초** — 상대 시간으로 신선도를 정한다

Expires — 초 대신 실제 만료 날짜

1 상대 시간

```
Cache-Control: max-age=484200
```

받은 시점부터 484200초 동안 신선

2 절대 날짜

deprecated

```
Expires: Fri, 05 Jul 2002, 05:00:00 GMT
```

이 날짜·시각이 지나면 만료

왜 권장 안 하나

절대 날짜는 **서버와 클라이언트 시계가 맞아야** 제대로 동작
많은 서버의 시계가 동기화돼 있지 않거나 부정확 → 만료 판단이 틀어짐

→ Expires는 더 이상 쓰지 않기를 권함 — 상대 시간인 max-age를 쓴다

must-revalidate — 만료되면 반드시 재검사

신선하지 않은 사본은 **원 서버와 재검사** 없이는 제공하면 안 됨 · 신선한 사본은 자유롭게 제공

	저장	신선할 때	만료됐을 때
`no-store`	✕ 아예 저장 안 함	-	-
`no-cache`	○	매번 재검사	매번 재검사
`must-revalidate`	○	재검사 없이 바로 사용	반드시 재검사

→ 신선할 땐 **그냥 쓰고**, 만료되면 **원 서버 재검사 없이는 절대 안 준다**

그럼 기본 동작과 뭐가 다른가

HTTP는 원래 **만료된 사본을 그냥 내주는 것**을 어느 정도 허용 — 차이는 **예외 상황**에서만 드러남

기본 동작이 허용하는 예외

1. **원 서버가 죽었거나 연결 안 됨**
   에러 대신 옛날 사본이라도 줌
2. **클라이언트가 오래된 것도 OK**
   `Cache-Control: max-stale`
3. **캐시가 성능을 위해 그렇게 설정됨**
   신선하지 않은 객체를 제공하도록

`must-revalidate` = 이런 예외를 전부 금지

	서버 정상	서버 연결 안 됨
기본	재검사	만료된 사본 줄 수 있음
`must-revalidate`	재검사	504 Gateway Timeout

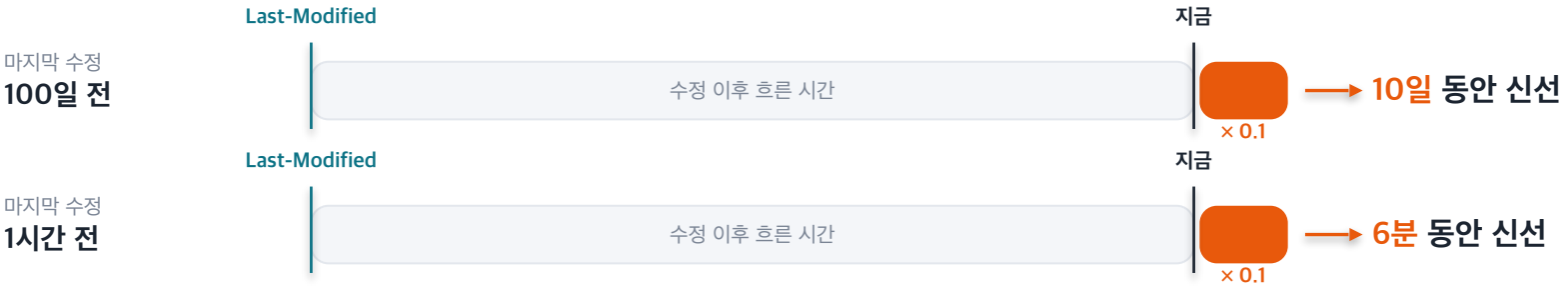
예 **계좌 잔액**처럼 틀린 값보다 에러가 나은 데이터

→ 서버가 죽었으면 옛날 사본 대신 504 — 틀린 값보다 에러가 낫다

만료 정보가 없으면 — 캐시가 짐작한다

`max-age` · `Expires` 둘 다 없음 → 언제까지 써도 되는지 모름 → `Last-Modified`를 보고 짐작

신선 기간 = (지금 − `Last-Modified`) × **LM 인자** 예) LM 인자 = 0.1



→ 교훈: 캐시 정책은 추측에 맡기지 말고 Cache-Control로 직접 정해주자

클라이언트도 신선도를 요구할 수 있다

Cache-Control은 응답 헤더로도, 요청 헤더로도 쓸 수 있다 — 이번엔 클라이언트가 보내는 쪽



1 no-cache

재검사 전에는 캐시된 리소스를 받지 않는다

캐시 사본 → 반드시 재검사

2 min-fresh=<s>

지금부터 적어도 s초 후까지 신선한 문서만 받는다

곧 만료될 사본도 거절

3 max-stale=<s>

신선하지 않은 문서도 받는다

s 지정 시: 만료 후 s초까지 OK

→ 서버만 정하는 게 아니다 — 클라이언트도 요청 헤더로 '이 정도로 신선한 것만' 요구

부록 · 7.9.6 클라이언트 신선도 제약

새로고침 버튼이 실제로 이 헤더를 보낸다

리프레시 버튼 = 요청 헤더가 추가된 GET → 강제로 재검사하거나 서버에서 새로 받아옴

1 일반 새로고침 F5

```
GET /app.js HTTP/1.1
Cache-Control: max-age=0
If-None-Match: "v1"
```

→ **304 Not Modified** 캐시된 게 있으면 '바뀌었어?' 재검사

2 강력 새로고침 Ctrl+Shift+R

```
GET /app.js HTTP/1.1
Cache-Control: no-cache
Pragma: no-cache
```

→ **200 OK** 서버에서 새로 받아옴

직접 확인 개발자 도구 Network 탭 → 요청 헤더

Pragma: no-cache HTTP/1.0 시절 헤더 · 호환용, 지금은 의미 거의 없음

→ '배포했는데 안 바뀌어요' → 강력 새로고침부터 — 캐시를 건너뛰고 서버에서 새로 받는다

PART 2

캐시 제어 설정

7.10 · p.213 ~ 216

캐시 제어는 서버가 응답 헤더로 한다

책 (아파치 · HTML)	지금
mod_headers 응답 헤더를 직접 지정	NGINX `add_header Cache-Control …`
mod_expires 만료 시간을 자동으로 붙임	NGINX `expires 30d;`
<META HTTP-EQUIV="Cache-control" …> HTML 안에 캐시 지시	× 믿지 말 것 → 서버 헤더로

HTTP-EQUIV는 대부분의 브라우저·프록시가 무시 — 책의 결론도 '서버 헤더로 하라'

→ 이 한 줄만: **캐시 제어는 서버가 응답 헤더로 한다**

Spring에서는 이렇게

1 API 응답 — ResponseEntity

```
return ResponseEntity.ok()
    .cacheControl(CacheControl.maxAge(
        Duration.ofMinutes(10)))
    .body(data);
```

noCache() · noStore() — 배운 지시자 그대로

2 정적 파일 — application.yml

```
spring.web.resources.cache:
  cachecontrol:
    max-age: 365d
```

NGINX가 정적 파일을 서빙하면 → NGINX 설정에서

3 Spring Security 기본 동작 — 모든 응답에 자동으로

```
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
```

로그인 정보 같은 **민감한 응답**이
캐시되지 않게

→ '캐시 설정했는데 왜 안 되지?' → Security가 덮어쓰고 있는지 먼저 의심

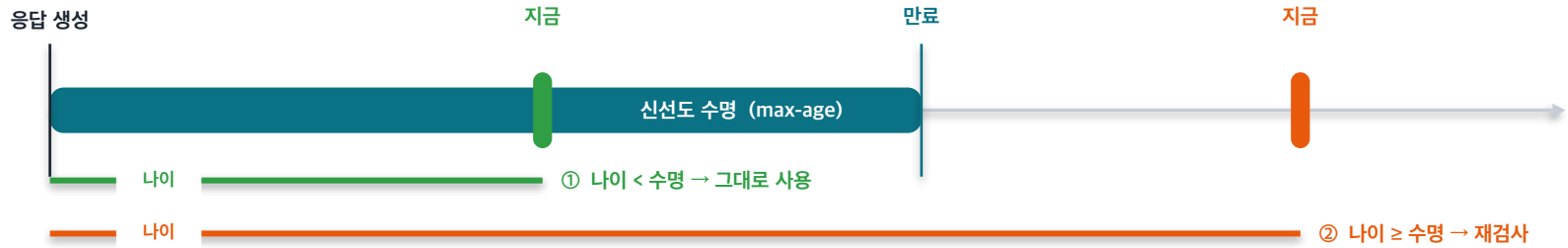
PART 3

자세한 알고리즘

7.11 · p.216 ~ 223

핵심은 한 줄 — 나이 < 신선도 수명

나이 < 신선도 수명 → 신선 · 그대로 사용　　아니면 → 서버에 재검사



신선도 수명

서버가 정해 줌 — 주로 `max-age`

옛날 방식: `Expires`

나이

응답이 만들어진 뒤 지난 시간

중간 프록시를 거치면 `Age` 헤더로 전달

덤 · 둘 다 안 보내면

`Last-Modified` × 약 10%로 추측

'헤더 안 줬는데 왜 캐시되지?' (9쪽)

→ 공식 자체는 **캐시 서버·프록시를 만드는 사람용** — 백엔드는 이 한 줄이면 충분